

RESUMO COMPLETO DE IA

Inteligência Artificial — Guia Didático para Prova

Com intuição, analogias, fórmulas explicadas e exemplos resolvidos

ÍNDICE

	Regressão Linear
1.	$h(x)$, $J(\theta)$, Batch GD, SGD, Forma Fechada
	Regressão Logística
2.	Sigmoid, Log-verossimilhança, GD Ascendente
	Perceptron
3.	Regra de atualização, Convergência, Limitações
	MLP — Redes Neurais
4.	Arquitetura, Feedforward, Backprop, Funções de ativação
	Avaliação de Classificadores
5.	Matriz de Confusão, Precisão, Recall, F1, ROC, AUC
	Pré-processamento
6.	Normalização, One-Hot, Missing Values, Discretização

1. REGRESSÃO LINEAR

Intuição: Imagine prever o preço de um apartamento pela área em m². Você risca uma reta num gráfico que "passa no meio" dos pontos. Regressão Linear é exatamente isso: encontrar a melhor reta (ou plano em múltiplas dimensões) que descreve a relação entre entrada (área) e saída (preço). O modelo é $h(x) = \theta_0 + \theta_1 \cdot x$, onde θ_0 e θ_1 são aprendidos.

■ O que é a hipótese $h(x)$?

A hipótese $h(x)$ é a **previsão do modelo** para uma entrada x . É uma combinação linear de todos os atributos. θ_0 é o intercepto (valor previsto quando todos atributos são zero). Cada θ_i é o peso que o modelo aprende para o atributo x_i .

$$h(x) = \theta_0 + \theta_1 \cdot x_1 + \dots + \theta_n \cdot x_n$$

Forma vetorial: $h(x) = \theta^T \cdot x$ (produto interno)

$h(x)$	valor previsto pelo modelo para a entrada x
θ_0	bias/intercepto — desloca a reta verticalmente
θ_i	peso do i -ésimo atributo — importância de x_i
$\theta^T \cdot x$	multiplicação vetor-vetor: soma de $\theta_i \cdot x_i$

■ Função de Custo $J(\theta)$ — Medindo o Erro

Precisamos medir o quão ruim está o modelo. $J(\theta)$ calcula o erro quadrático médio (MSE): eleva o erro ao quadrado (penaliza erros grandes mais que pequenos; evita cancelamento +/-). Nosso objetivo é minimizar $J(\theta)$.

Analogia: Arqueiro e o alvo

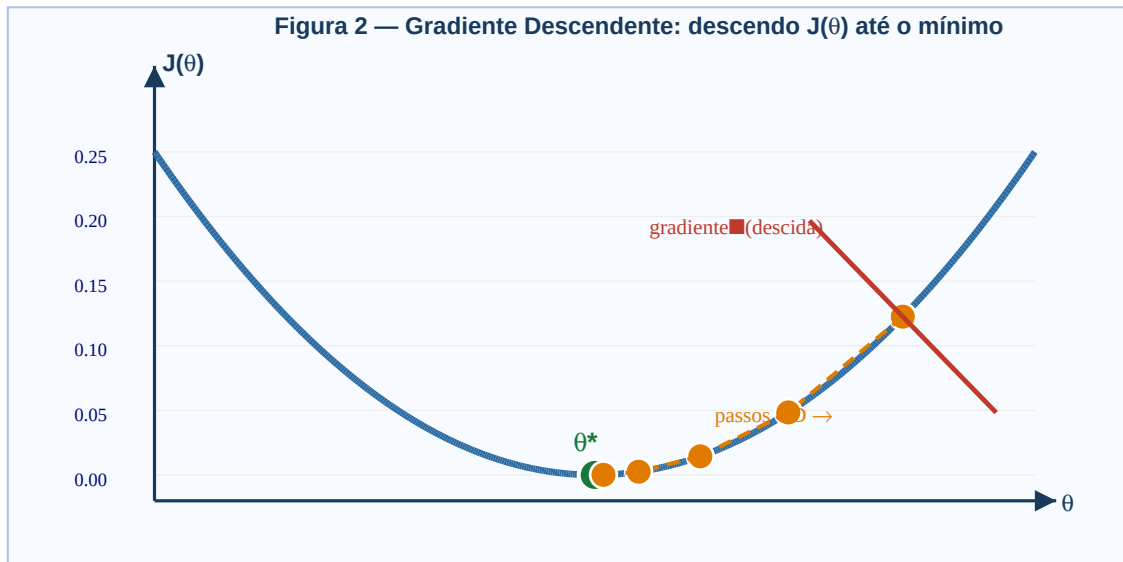
Cada exemplo é um tiro. O erro é a distância entre a flecha ($h(x)$) e o centro (y). $J(\theta)$ é a média dos quadrados das distâncias. Ajustamos a mira (θ) para minimizar esse erro.

$$J(\theta) = (1/2m) \cdot \sum_{i=1..m} [h(x^{(i)}) - y^{(i)}]^2$$

m	total de exemplos de treinamento
$h(x^{(i)})$	previsão para o i -ésimo exemplo
$y^{(i)}$	valor real (correto) do i -ésimo exemplo
$[...]^2$	eleva ao quadrado: penaliza erros grandes, evita cancelamento
$1/2$	constante de conveniência — cancela o 2 da derivada

■ Gradiente Descendente — Encontrando os melhores θ

$J(\theta)$ é como uma colina. Queremos chegar ao vale (mínimo). A cada passo, calculamos a inclinação (gradiente) e damos um passo na descida. Repetimos até convergir.



Curva $J(\theta)$ convexa. Gradiente = inclinação da tangente (vermelho). Pontos laranjas = passos do GD descendo até θ^* (verde = mínimo).

■ Batch Gradient Descent

Usa **todos os m exemplos** para calcular o gradiente antes de atualizar θ . Mais estável mas mais lento por iteração quando m é grande.

$$\theta_j \leftarrow \theta_j - \alpha \cdot (1/m) \cdot \sum_{i=1..m} [h(x^{(i)}) - y^{(i)}] \cdot x_j^{(i)}$$

α (alpha)	taxa de aprendizado — tamanho do passo. Muito grande: oscila. Pequeno: demora.
$(1/m) \cdot \sum$	gradiente médio sobre todos m exemplos
$h - y$	erro do modelo no exemplo i (previsão menos real)
$x_j^{(i)}$	valor do j -ésimo atributo do i -ésimo exemplo

■ SGD — Stochastic Gradient Descent

Usa **1 exemplo por vez**, escolhido aleatoriamente. Muito mais rápido por iteração, mas o caminho até o mínimo oscila bastante. Na prática, converge bem para grandes conjuntos de dados.

Para cada exemplo $(x^{(i)}, y^{(i)})$ — um de cada vez:

$$\theta_j \leftarrow \theta_j - \alpha \cdot [h(x^{(i)}) - y^{(i)}] \cdot x_j^{(i)}$$

IMPORTANTE: Batch GD: usa todos m exemplos por atualização. SGD: usa 1 exemplo. Mini-Batch: usa k exemplos (mais comum na prática).

■ Forma Fechada — Solução Analítica Exata

Para regressão linear existe solução matemática direta: derivamos $J(\theta)$, igualamos a zero e resolvemos. Calcula θ ótimo de uma vez, sem iterações. **Desvantagem:** inverter matriz custa $O(n^3)$ — para n grande, prefira GD.

$$\theta = (X^T \cdot X)^{-1} \cdot X^T \cdot y$$

X	matriz de design $m \times (n+1)$: cada linha = $[1, x_1, x_2, \dots, x_n]$
y	vetor com todos os valores reais, dimensão $m \times 1$
$(X^T X)^{-1}$	inversa — existe só se colunas de X forem linearmente independentes

Exemplo Resolvido: Regressão Linear — 1 passo de GD

Dados: $x=[1, 2, 3, 4]$, $y=[2, 4, 5, 4]$ $\theta_0=1$, $\theta_1=1$, $\alpha=0.1$

■■ Previsões e erros ■■

$h(1)=2$ (erro=0) $h(2)=3$ (erro=-1) $h(3)=4$ (erro=-1) $h(4)=5$ (erro=+1)

$J(\theta) = (1/8) \cdot [0+1+1+1] = 0.375$

■■ Atualização de θ_1 (Batch GD) ■■

Gradiente = $(1/4) \cdot [0 \cdot 1 + (-1) \cdot 2 + (-1) \cdot 3 + 1 \cdot 4] = (1/4) \cdot (-1) = -0.25$

$\theta_1 \leftarrow 1 - 0.1 \cdot (-0.25) = 1.025$ (aumentou pois gradiente era negativo)

2. REGRESSÃO LOGÍSTICA

Intuição: E se em vez de prever um número, você quer classificar: "spam ou não spam?" Regressão Linear não serve — pode prever valores fora de $[0,1]$. A Regressão Logística "espreme" a saída pela função sigmoid para que fique sempre em $(0,1)$, interpretável como probabilidade. $h(x) \geq 0.5 \rightarrow$ classe 1. $h(x) < 0.5 \rightarrow$ classe 0.

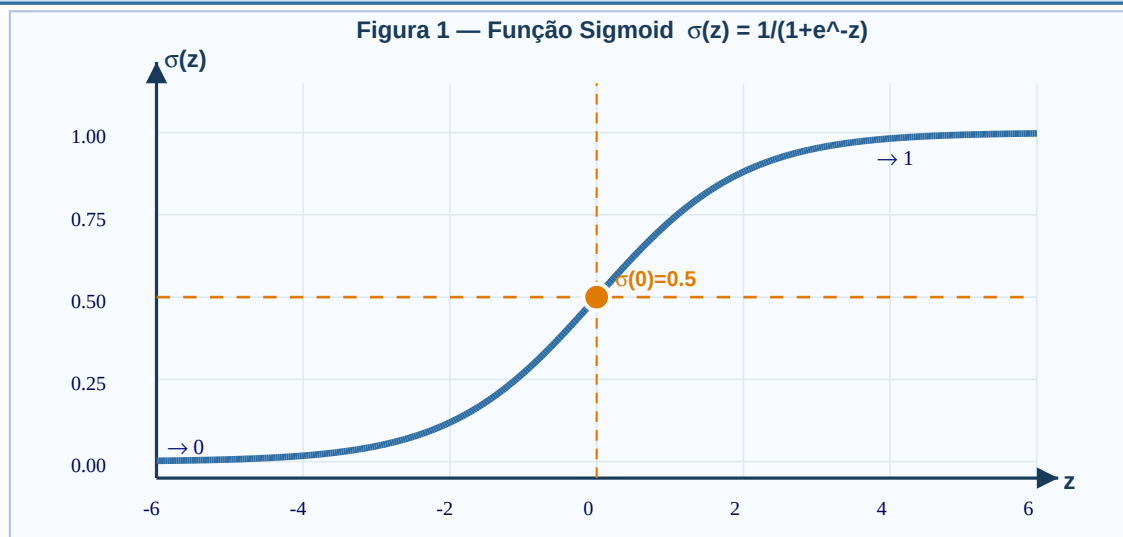
■ A Função Sigmoid $\sigma(z)$

A sigmoid é uma curva em "S" que transforma qualquer número real z em uma probabilidade entre 0 e 1. Muito positivo $\rightarrow$ próximo de 1. Muito negativo $\rightarrow$ próximo de 0. $\sigma(0) = 0.5$ = ponto de equilíbrio.

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

$$\text{Hipótese: } h(x) = \sigma(\theta^T \cdot x) = 1 / (1 + e^{-(\theta_0 + \theta_1 \cdot x_1 + \dots)})$$

Interpretação: $h(x) = P(y=1 \mid x; \theta)$ — probabilidade de ser classe 1



Curva sigmoid em S. Assíntota inferior em 0, superior em 1. Ponto laranja: $\sigma(0)=0.5$ (limiar de decisão). Linha tracejada laranja: $z=0$, equivale a $h(x)=0.5$.

$z = \theta^T \cdot x$	logit: combinação linear dos atributos
$e^{(-z)}$	exponencial negativa — define a forma de S da curva
$h(x)$	probabilidade de $y=1$, sempre em $(0,1)$
Decisão	$y=1$ se $h(x) \geq 0.5$, equivalente a $\theta^T \cdot x \geq 0$

■ Por que não MSE? — Log-Verossimilhança

MSE com sigmoid cria $J(\theta)$ não-convexa (vários mínimos locais — ruim para GD). Em vez disso, maximizamos a **log-verossimilhança**: queremos parâmetros θ que tornem os dados observados mais prováveis. Para $y=1$: queremos $h(x)$ grande. Para $y=0$: queremos $h(x)$ pequeno.

Analogia: Apostas e probabilidades

Você quer θ que torna os dados observados "mais prováveis". Se um aluno se formou ($y=1$) e o modelo diz $P(\text{formou})=0.95$, ótimo! Se diz 0.10, ruim — ajustamos θ . A verossimilhança é o produto dessas probabilidades.

$L(\theta) = \prod h(x^i)^{y^i} \cdot (1-h(x^i))^{(1-y^i)}$ [produto – maximizar]

$\ell(\theta) = \sum [y^i \cdot \log(h(x^i)) + (1-y^i) \cdot \log(1-h(x^i))]$ [log – mais estável]

Quando $y=1$: $\text{contribuicao} = \log(h(x)) \rightarrow$ queremos $h(x)$ próximo de 1

Quando $y=0$: $\text{contribuicao} = \log(1-h(x)) \rightarrow$ queremos $h(x)$ próximo de 0

$L(\theta)$	produto das probabilidades — maximizar. Quanto maior, melhor o ajuste.
$\ell(\theta)$	log da verossimilhança — transforma produto em soma, numericamente estável
$\log(h(x))$	sempre ≤ 0 . Quanto mais próximo de 0, melhor. $\log(1)=0$ = perfeito.

■ Gradiente Ascendente — Treinando o Modelo

Para maximizar $\ell(\theta)$, fazemos gradiente **ascendente** (sinal +). A fórmula tem a mesma forma que o GD linear — só muda o sinal e a definição de $h(x)$.

$\theta_j \leftarrow \theta_j + \alpha \cdot (1/m) \cdot \sum_{i=1..m} [y^i - h(x^i)] \cdot x_j^i$

Erro = $y - h(x)$: +1 se perdeu positivo (h baixo), -1 se alarme falso (h alto)

IMPORTANTE: Sinal + porque MAXIMIZAMOS $\ell(\theta)$. Em regressão linear era – porque MINIMIZÁVAMOS $J(\theta)$. A forma da fórmula é idêntica — só o sinal e $h(x)$ mudam!

Exemplo Resolvido: Regressão Logística — Questão da Prova

$\theta_0 = -19, \theta_1 = 0.05 \mid x = [200, 300, 800, 600], y = [0, 0, 1, 1]$

■ ■ Calculando $h(x) = \sigma(\theta_0 + \theta_1 \cdot x)$ ■ ■

$x=200$: $z = -19 + 0.05 \cdot 200 = -9.0 \rightarrow h = 1/(1+e^9) \approx 0.000123$

$x=300$: $z = -19 + 0.05 \cdot 300 = -4.0 \rightarrow h = 1/(1+e^4) \approx 0.01799$

$x=800$: $z = -19 + 0.05 \cdot 800 = +21 \rightarrow h = 1/(1+e^{-21}) \approx 0.99999$

$x=600$: $z = -19 + 0.05 \cdot 600 = +11 \rightarrow h = 1/(1+e^{-11}) \approx 0.99998$

■ ■ Log-verossimilhança ■ ■

$y=0$: $\log(1-0.000123) \approx -0.000123$ $y=0$: $\log(1-0.01799) \approx -0.01816$

$y=1$: $\log(0.99999) \approx -0.00001$ $y=1$: $\log(0.99998) \approx -0.00002$

$\ell(\theta) \approx -0.0183 \rightarrow L(\theta) = e^{(-0.0183)} \approx 0.9818$ (próximo de 1 = bom!)

■ ■ GD (1 passo, $\alpha=0.1$, exemplo $x=200, y=0$) ■ ■

$$\text{erro} = y - h(x) = 0 - 0.000123 = -0.000123$$

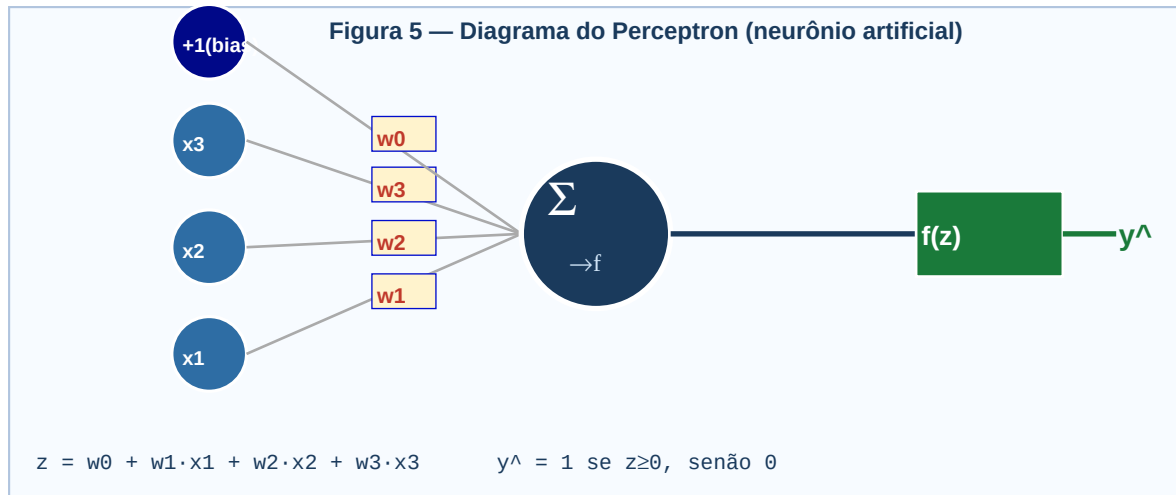
$$\theta_0 \leftarrow -19 + 0.1 \cdot (-0.000123) \cdot 1 \approx -19.0000123$$

$$\theta_1 \leftarrow 0.05 + 0.1 \cdot (-0.000123) \cdot 200 = 0.04754$$

3. PERCEPTRON

Intuição: O Perceptron é o neurônio artificial mais simples. Recebe várias entradas, multiplica cada uma por um peso, soma tudo. Se a soma for ≥ 0 : dispara (saída 1). Se for < 0 : não dispara (saída 0). Como um interruptor de luz: ou acende ou não. Aprende ajustando os pesos para classificar exemplos corretamente.

■ Diagrama do Neurônio



Entradas $x_1..x_3$ chegam ponderadas pelos pesos $w_1..w_3$. +1 cinza = bias (w_0). Neurônio azul soma tudo: $z = \sum w_i \cdot x_i$. Caixa verde: ativação degrau $\rightarrow$ saída $y^{\wedge}$.

■ Função de Ativação Limiar (Step Function)

Diferente da sigmoid, o Perceptron usa uma função degrau: salta abruptamente de 0 para 1 no limiar $z=0$. Não há probabilidade — é uma decisão binária definitiva.

$$z = \theta_0 + \theta_1 \cdot x_1 + \dots + \theta_n \cdot x_n$$

$$f(x) = 1 \text{ se } z \geq \theta \text{ (dispara - classe positiva)}$$

$$f(x) = 0 \text{ se } z < \theta \text{ (não dispara - classe negativa)}$$

■ Como o Perceptron Aprende — Regra de Atualização

Passa um exemplo, olha o resultado, e se errou, ajusta os pesos. Se previu 0 mas era 1 (falso negativo): aumenta pesos. Se previu 1 mas era 0 (falso positivo): diminui pesos. Se acertou: não faz nada.

- | | |
|---|---|
| 1 | Passe um exemplo ($x^{\wedge}(i)$, $y^{\wedge}(i)$)
Calcule $z = \theta^T \cdot x$ e obtenha $f(x)$ |
| 2 | Calcule o erro
$\text{erro} = y^{\wedge}(i) - f(x^{\wedge}(i))$. Valores: +1, -1, ou 0 |
| 3 | Atualize cada peso
$\theta_j \leftarrow \theta_j + \alpha \cdot (y^{\wedge}(i) - f(x^{\wedge}(i))) \cdot x_j^{\wedge}(i)$ para todo j |

4	Repita para todos os exemplos Uma passagem completa = 1 época (epoch)
5	Repita até convergir Para quando não houver mais erros de classificação

$\theta_j \leftarrow \theta_j + \alpha \cdot (y^{(i)} - f(x^{(i)})) \cdot x_j^{(i)}$	
y=1, f=0 (perdeu): $\theta_j \leftarrow \theta_j + \alpha \cdot x_j$ (pesos sobem $\rightarrow$ z sobe $\rightarrow$ mais chance de f=1)	
y=0, f=1 (alarme): $\theta_j \leftarrow \theta_j - \alpha \cdot x_j$ (pesos descem $\rightarrow$ z desce $\rightarrow$ mais chance de f=0)	
Acertou: $\theta_j \leftarrow \theta_j$ (sem alteração)	
α (alpha)	taxa de aprendizado — geralmente 1 no Perceptron clássico
$y^{(i)} - f(x^{(i)})$	erro: +1 (perdeu positivo), -1 (alarme falso), 0 (acertou)
$x_j^{(i)}$	valor do j-ésimo atributo do i-ésimo exemplo

■ Convergência e Limitação Principal

Teorema de Convergência: Se os dados forem **linearmente separáveis** (existe reta/plano separando as classes), o Perceptron **sempre converge** em número finito de passos. Caso contrário, oscila para sempre.

Limitação clássica — XOR: O XOR não é linearmente separável. Isso motivou o desenvolvimento do MLP — que resolve XOR combinando múltiplos perceptrons.

IMPORTANTE: Perceptron resolve SOMENTE problemas linearmente separáveis. Para qualquer coisa mais complexa (XOR, imagens, texto) $\rightarrow$ MLP.

Exemplo Resolvido: Perceptron — Atualização de pesos

$\theta_0=0, \theta_1=0, \theta_2=0, \alpha=1$

Exemplo $x=[1,3], y=0$:

$z = 0 + 0 \cdot 1 + 0 \cdot 3 = 0 \rightarrow f(x)=1 (z \geq 0)$

ERRO! $y=0$ mas previu 1 $\rightarrow$ erro = $0-1 = -1$

$\theta_0 \leftarrow 0 + 1 \cdot (-1) \cdot 1 = -1$

$\theta_1 \leftarrow 0 + 1 \cdot (-1) \cdot 1 = -1$

$\theta_2 \leftarrow 0 + 1 \cdot (-1) \cdot 3 = -3$

Verificação: $z = -1 + (-1) \cdot 1 + (-3) \cdot 3 = -1 - 1 - 9 = -11 < 0 \rightarrow f=0 \checkmark$

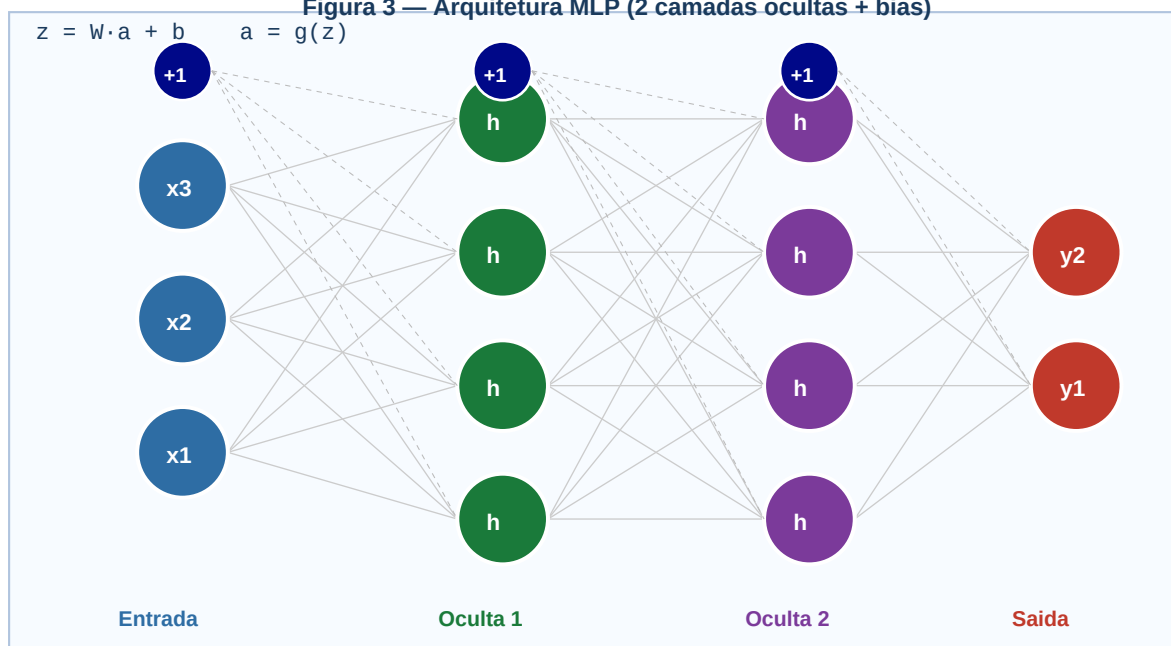
4. MLP — REDES NEURAIS MULTICAMADAS

Intuição: O MLP é um cérebro com neurônios em camadas. Cada camada aprende representações mais abstratas: numa rede de reconhecimento de rostos, a 1ª aprende bordas, a 2ª aprende olhos/nariz, a 3ª aprende rostos completos. A "mágica" está nas ativações não-lineares: sem elas, empilhar camadas produziria apenas outra função linear — equivalente a uma única camada.

■ Arquitetura da Rede

Uma MLP tem: **camada de entrada** (dados brutos), uma ou mais **camadas ocultas** (representações intermediárias) e **camada de saída** (previsão final). Cada neurônio conecta-se a **todos** os da próxima camada. Cada conexão tem peso W e cada neurônio tem bias b .

Figura 3 — Arquitetura MLP (2 camadas ocultas + bias)



3 entradas → 2 camadas ocultas (4 neurônios cada) → 2 saídas. Nós cinza (+1) = bias. Linhas cinzas = conexões ponderadas.

■ Feedforward — Propagação para Frente

Para fazer uma previsão, os dados fluem da entrada até a saída. Em cada camada: (1) calcular pré-ativação z^l , (2) aplicar ativação a^l . A ativação de uma camada é a entrada da próxima.

Pre-ativacao: $z^l[l] = W^l[l] \cdot a^{l-1} + b^l[l]$

Ativacao: $a^l[l] = g^l[l](z^l[l])$

$a^l[0] = x$ (entrada) | $y_{\text{hat}} = a^l[L]$ (saída final)

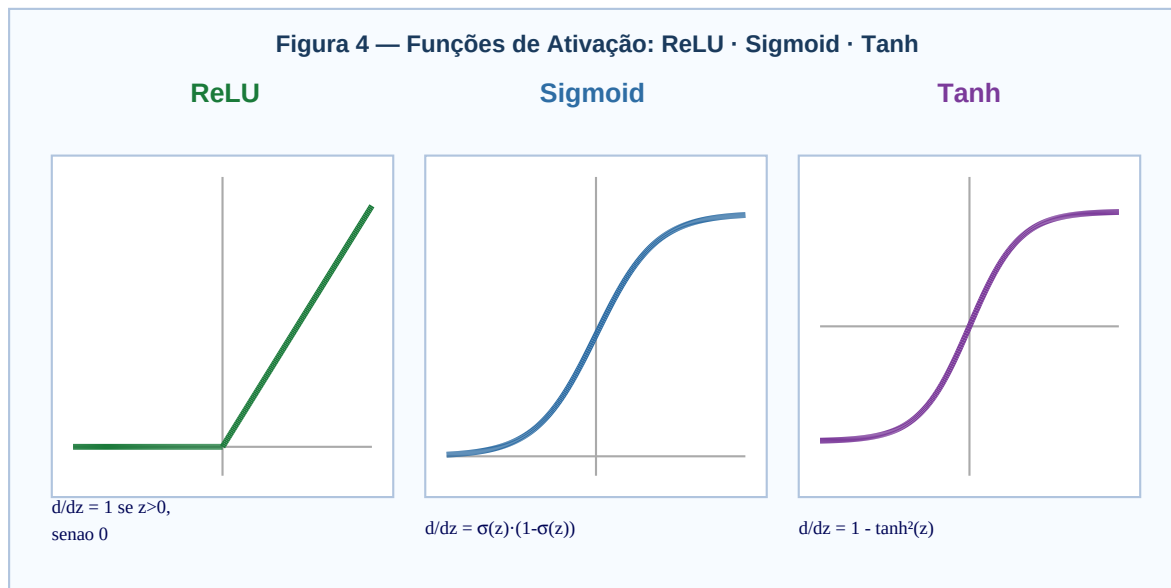
$W^l[l]$	matriz de pesos da camada l: dimensão (nós de l) × (nós de l-1)
$b^l[l]$	vetor de bias da camada l: dimensão (nós de l) × 1
a^{l-1}	ativações da camada anterior — entrada para a camada l
$g^l[l]$	função de ativação escolhida para a camada l

$z^l[i]$

pré-ativação linear antes de passar pela ativação

■ Funções de Ativação — Por que precisamos delas?

Sem ativações não-lineares, empilhar camadas produziria apenas outra função linear — seria o mesmo que uma única camada. As ativações introduzem **não-linearidade**, permitindo aprender padrões complexos como curvas e fronteiras não-lineares.



ReLU (verde): retifica negativos para 0. Sigmoid (azul): comprime para (0,1). Tanh (roxo): comprime para (-1,1). Abaixo: fórmula da derivada de cada ativação (usada no backpropagation).

Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$ **Derivada:** $\sigma(z) \cdot (1 - \sigma(z))$

ReLU: $\max(0, z)$ **Derivada:** 1 se $z > 0$; 0 se $z \leq 0$

Tanh: $(e^z - e^{-z}) / (e^z + e^{-z})$ **Derivada:** $1 - \tanh^2(z)$

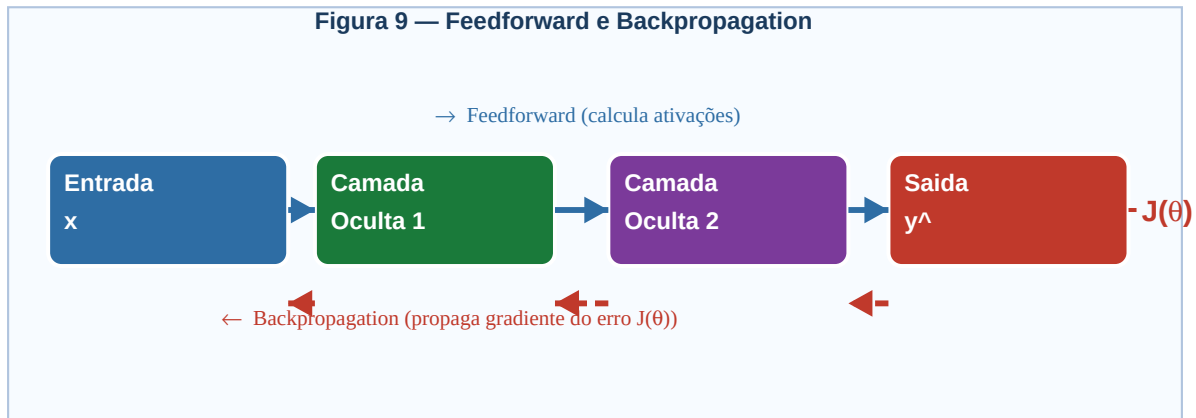
Softmax: $e^{z_j} / \sum e^{z_k}$ (multiclasse) **Saídas somam 1 (probabilidades)**

ReLU	Mais usada em ocultas. Resolve vanishing gradient. Simples e rápida.
Sigmoid	Boa para saída binária. Sofre vanishing gradient em redes profundas.
Tanh	Centrada em zero — geralmente melhor que sigmoid em camadas ocultas.
Softmax	Saída multiclasse — cada saída = probabilidade de uma classe.

■ Treinamento — Backpropagation

Após o feedforward, calculamos o erro na saída e propagamos esse erro de volta, calculando o gradiente de $J(\theta)$ em relação a cada peso pela regra da cadeia. Depois atualizamos todos os pesos com GD.

Figura 9 — Feedforward e Backpropagation



Setas azuis → feedforward (calcula ativações). Setas vermelhas tracejadas ← backpropagation (gradiente do erro $J(\theta)$ vai de volta).

1	Feedforward Calcule $a^l[l]$ para $l=1,\dots,L$ usando $z^l[l]=W^l[l]\cdot a^l[l-1]+b^l[l]$
2	Erro na saída $\delta^L[L] = a^L[L] - y$ (previsão menos real na saída)
3	Propagar erro $l=L-1,\dots,1$: $\delta^l[l] = (W^{l+1})^T \cdot \delta^{l+1} \odot g'(z^l[l])$
4	Calcular gradientes $\partial J / \partial W^l[l] = \delta^l[l] \cdot (a^{l-1})^T$ $\partial J / \partial b^l[l] = \delta^l[l]$
5	Atualizar pesos $W^l[l] \leftarrow W^l[l] - \alpha \cdot \partial J / \partial W^l[l]$

$\delta^l[l]$	gradiente local da camada l — contribuição de cada neurônio ao erro total
$\odot$	produto elemento a elemento (Hadamard) — multiplica posição a posição
$g'(z^l[l])$	derivada da ativação — mede sensibilidade da saída à pré-ativação
$(W^{l+1})^T$	transposta dos pesos da próxima camada — distribui o erro de volta

IMPORTANTE: Derivadas: sigmoid'= $\sigma(1-\sigma)$ | ReLU'=1 se $z>0$, 0 se $z\leq 0$ | tanh'= $1-\tanh^2$

Exemplo Resolvido: Feedforward MLP — 1 camada oculta

$x=[0.5, 0.8]$ $W^1[1]=\begin{bmatrix} 0.2 & 0.4 \\ 0.3 & 0.1 \end{bmatrix}$ $b^1[1]=[0, 0]$

■ Camada oculta (sigmoid) ■

$z1^1[1] = 0.2 \cdot 0.5 + 0.4 \cdot 0.8 = 0.42 \rightarrow a1^1[1] = \sigma(0.42) \approx 0.6034$

$z2^1[1] = 0.3 \cdot 0.5 + 0.1 \cdot 0.8 = 0.23 \rightarrow a2^1[1] = \sigma(0.23) \approx 0.5572$

■ Camada de saída (sigmoid) ■

$z^2[2] = 0.5 \cdot 0.6034 + 0.6 \cdot 0.5572 = 0.636 \rightarrow \text{Saída} = \sigma(0.636) \approx 0.654$

Interpretação: 65.4% de probabilidade para a classe 1.

5. AVALIAÇÃO DE CLASSIFICADORES

Intuição: Como saber se o classificador é bom? Acurácia parece óbvio, mas imagine diagnóstico de doença rara (1% dos casos): um modelo que sempre diz "saúdável" tem 99% de acurácia mas é inútil! Precisamos de métricas que diferenciam os **tipos** de erro.

■ Matriz de Confusão — A Base de Tudo

Organiza as previsões em 4 categorias comparando o que o modelo previu com o que era verdadeiro. Dá visão completa dos acertos e tipos de erros.

Figura 6 — Matriz de Confusão (Classificação Binária)

		Predito: POS	Predito: NEG
Real: ■ NEG		FP Falso Positivo	VN Verdadeiro Negativo
	Real: ■ POS	VP Verdadeiro Positivo	FN Falso Negativo

Verde = acertos (VP e VN). Vermelho = erros (FP e FN). FP = alarme falso (Erro Tipo I). FN = caso perdido (Erro Tipo II — mais perigoso em diagnósticos).

VP (True Positive)	Previu POSITIVO e era POSITIVO — acerto positivo
VN (True Negative)	Previu NEGATIVO e era NEGATIVO — acerto negativo
FP (False Positive)	Previu POSITIVO mas era NEGATIVO — "alarme falso", Erro Tipo I
FN (False Negative)	Previu NEGATIVO mas era POSITIVO — "caso perdido", Erro Tipo II (grave em medicina)

■ Métricas — Cada uma responde uma pergunta diferente

Acurácia: "De todos os exemplos, quantos o modelo acertou?" Enganosa em dados desbalanceados.

Precisão: "Dos que o modelo disse positivos, quantos eram realmente?" Alta precisão = poucos alarmes falsos. Importante quando FP é caro (spam).

Recall (Sensibilidade): "Dos positivos reais, quantos o modelo encontrou?" Alto recall = poucos casos perdidos. Crítico em diagnósticos!

F1-Score: Média harmônica de Precisão e Recall. Balanceia os dois quando você não pode priorizar um sobre o outro.

Acuracia = $(VP+VN) / (VP+VN+FP+FN)$

Precisao = $VP / (VP+FP)$ "dos que PREVI positivo, quantos ERAM?"

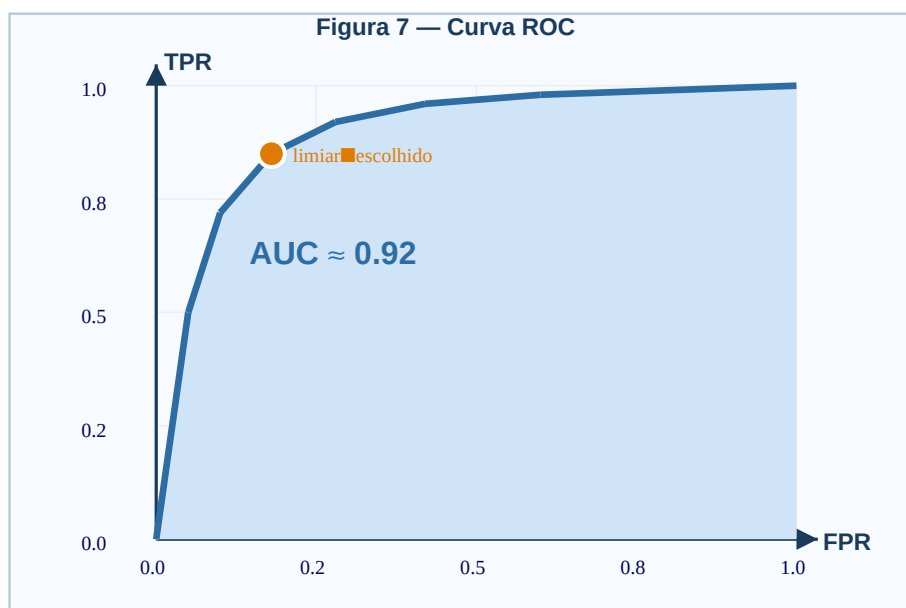
Recall = $VP / (VP+FN)$ "dos que ERAM positivos, quantos EU ACHEI?"

F1-Score = $2 \cdot (P \cdot R) / (P+R)$ media harmonica – equilibra P e R

Especif. = $VN / (VN+FP)$ taxa de negativos corretamente identificados

■ Curva ROC e AUC

A curva ROC mostra o trade-off entre Recall (TPR) e FPR para todos os limiares de decisão (de 0 a 1). Uma curva que abraça o canto superior esquerdo é melhor. AUC resume tudo em um número: quanto mais próximo de 1, melhor.



Eixo Y = TPR (Recall). Eixo X = FPR (1-Especificidade). Diagonal = classificador aleatório (AUC=0.5). Curva azul = bom classificador (AUC≈0.92). Área azul = AUC. Ponto laranja = limiar de operação escolhido.

AUC = area sob a curva ROC

AUC = 1.0 → perfeito **AUC = 0.5** → inútil **AUC > 0.5** → melhor que aleatório

Interpretacao: probabilidade de ranquear um positivo acima de um negativo.

■ Métodos de Avaliação — Como dividir os dados?

Não podemos avaliar no mesmo conjunto em que treinamos — o modelo memorizou esses dados. Precisamos de dados não vistos durante o treino.

1 Holdout
70-80% treino, 20-30% teste. Simples mas dependente do sorteio da divisão.

2 k-Fold Cross-Validation
k partes iguais. Treina em k-1, testa na k-ésima. Repete k vezes. Típico: k=10.

3

Leave-One-Out (LOO)

k-Fold com $k=m$: deixa 1 exemplo de fora por vez. Custoso computacionalmente.

4

Bootstrap

Amostragem com reposição. ~63.2% dos dados no treino. Restante = out-of-bag (teste).

Exemplo Resolvido: Calculando todas as métricas

VP=50, FN=10, FP=5, VN=35 (total=100)

Acuracia = $(50+35)/100 = 0.850 = 85.0\%$

Precisao = $50/(50+5) = 50/55 \approx 0.909 = 90.9\%$ (poucos alarmes falsos)

Recall = $50/(50+10) = 50/60 \approx 0.833 = 83.3\%$ (perde 10 de 60 positivos)

F1-Score = $2 \cdot (0.909 \cdot 0.833) / (0.909 + 0.833) \approx 0.869 = 86.9\%$

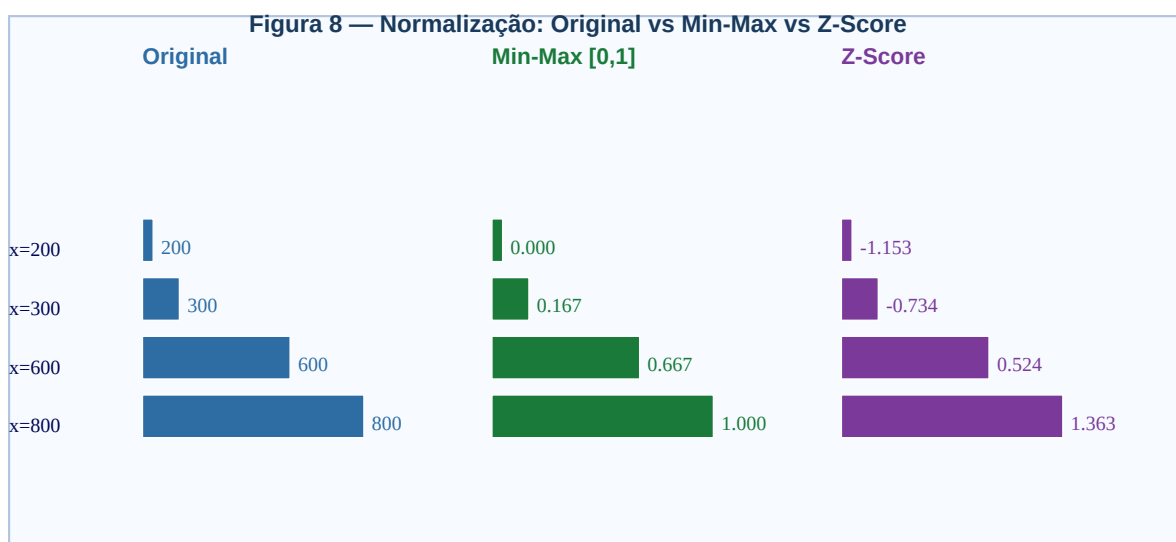
Especif. = $35/(35+5) = 35/40 = 0.875 = 87.5\%$

6. PRÉ-PROCESSAMENTO

Intuição: Dados do mundo real são bagunçados: valores faltando, escalas muito diferentes (salário em reais vs. idade em anos), variáveis categóricas (cidade, cor), outliers. Pré-processamento limpa e organiza os dados antes de treinar qualquer modelo. É a etapa que mais impacta o desempenho na prática — lixo entra, lixo sai!

■ Normalização — Por que é necessária?

Imagine dois atributos: salário (1.000 a 50.000) e idade (18 a 65). No GD, o atributo de maior escala vai dominar os pesos. Normalização coloca todos na mesma escala, permitindo que o GD convirja muito mais rápido e de forma mais estável.



Dados [200,300,600,800] nas três representações. Original: escala grande. Min-Max: range [0,1]. Z-Score: média=0, std=1.

Min-Max: $x' = (x - x_{\min}) / (x_{\max} - x_{\min})$ resultado em [0, 1]

Sensível a outliers (um outlier distorce tudo).

Z-Score: $x' = (x - \mu) / \sigma$ média=0, desvio=1

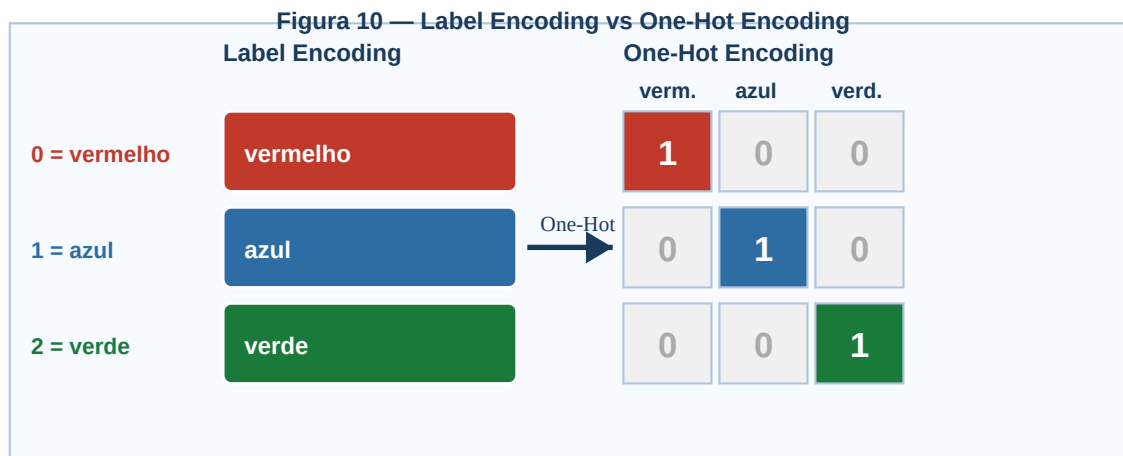
Robusto a outliers. Assume distribuição aproximadamente normal.

x_min, x_max	mínimo e máximo — calculados SOMENTE no treino
μ (mu)	média aritmética do atributo no treino
σ (sigma)	desvio padrão do atributo no treino
x'	valor transformado — substitui o original na entrada do modelo

IMPORTANTE: Calcule μ , σ , min, max SOMENTE no conjunto de TREINO. Aplique esses valores ao teste. Usar estatísticas do teste = data leakage!

■ Codificação de Variáveis Categóricas

Modelos de ML trabalham com números. **Label Encoding** atribui um inteiro a cada categoria (0,1,2). Problema: o modelo interpreta que verde (2) > azul (1) > vermelho (0) — ordem artificial! **One-Hot Encoding** cria uma coluna binária por categoria, sem ordem.



Esquerda: Label Encoding — simples mas introduz ordem artificial. Setas central → One-Hot. Direita: grid One-Hot — coluna da categoria correta recebe 1, demais recebem 0.

Label: vermelho=0, azul=1, verde=2 (1 coluna)

One-Hot: vermelho=[1,0,0] azul=[0,1,0] verde=[0,0,1] (3 colunas)

Para K categorias: cria K colunas (ou K-1 para evitar multicolinearidade).

■ Valores Ausentes (Missing Values)

Dados do mundo real frequentemente têm NaN. Não podemos simplesmente ignorar — perderíamos dados preciosos.

1	Remoção Deletar linhas/colunas com muitos NaN (só se < 5% dos dados e ausência aleatória)
2	Média (μ) Substituir NaN pela média do atributo — bom para numéricos sem outliers
3	Mediana Substituir pela mediana — melhor quando há outliers (mais robusta)
4	Moda Substituir pelo valor mais frequente — padrão para atributos categóricos
5	KNN/Modelo Prever o valor faltante com um modelo — mais sofisticado e preciso

■ Discretização — Contínuo para Discreto

Converte atributos contínuos em intervalos discretos. Útil para algoritmos que exigem entrada discreta (ex: Naive Bayes categórico).

Equal-Width: amplitude = (max-min)/k (intervalos de mesma largura)

Equal-Frequency: mesmo número de exemplos por intervalo (útil para dados assimétricos)

Por Entropia: usa os rótulos y para encontrar os melhores pontos de corte (supervisionado)

Exemplo Resolvido: Normalização — Dados da Prova

Notas ENEM: $x=[200, 300, 600, 800]$

■ ■ Min-Max ($x_{\min}=200, x_{\max}=800, \text{range}=600$) ■ ■

$$x'(200) = (200-200)/600 = 0.000$$

$$x'(300) = (300-200)/600 \approx 0.167$$

$$x'(600) = (600-200)/600 \approx 0.667$$

$$x'(800) = (800-200)/600 = 1.000$$

■ ■ Z-Score ($\mu=475, \sigma \approx 238.5$) ■ ■

$$\mu = (200+300+600+800)/4 = 475$$

$$\sigma = \sqrt{[(200-475)^2 + (300-475)^2 + (600-475)^2 + (800-475)^2]/4} \approx 238.5$$

$$x'(200) = (200-475)/238.5 \approx -1.153$$

$$x'(300) = (300-475)/238.5 \approx -0.734$$

$$x'(600) = (600-475)/238.5 \approx 0.524$$

$$x'(800) = (800-475)/238.5 \approx 1.363$$

Verificação: média dos z-scores = $(-1.153-0.734+0.524+1.363)/4 \approx 0$ ✓